

Track Insights — Build Your First Page

Developer onboarding · 📅 Sep 21, 2026 · @Someone

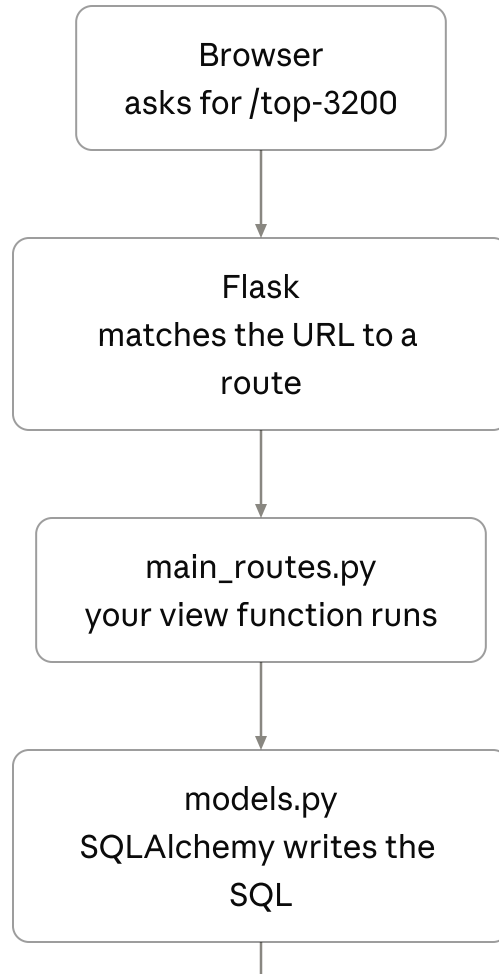
What you're building

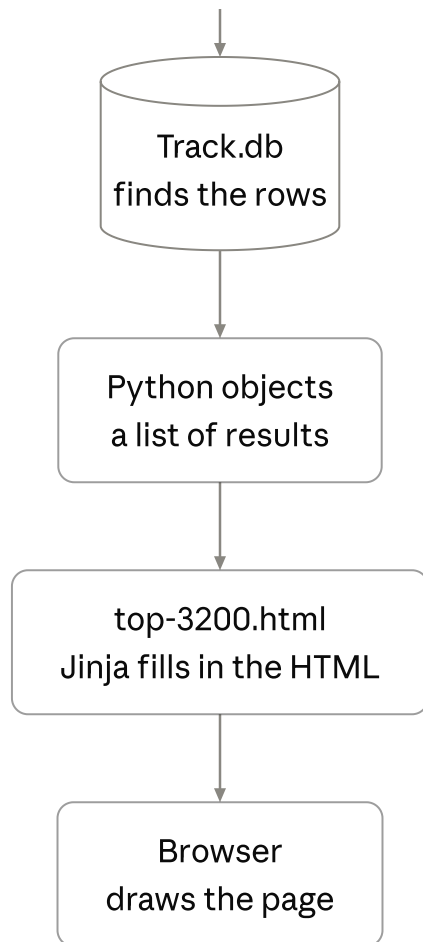
By the end you will have added a real page to Track Insights, running on your own laptop, with your name on the commit: the ten fastest boys' 3200 Meters times in our database, with athlete, school, time and meet.

It is small on purpose. One small page makes you touch every layer of the app, in order, so that afterwards you can draw what happens between a click and a page. Every other task on this project is a variation on that. Plan on two sessions: Parts 1-3, then Parts 4-7.

The big picture: what happens when someone clicks a link

A web app is a program that sits and waits. A browser asks it a question, it answers, and it goes back to waiting. Here is the full round trip for one page load:





Read it as a sentence: the browser asks for a URL, Flask decides which function handles that URL, the function asks the database for data, the database hands back rows, the template turns those rows into HTML, and the browser draws it.

1. **Every page works this way.** Different queries, different templates, same seven steps. Some pages run it twice: they send a nearly empty page, then JavaScript asks an `/api/...` URL for the data and fills the table in.
2. **The browser never touches the database.** It only ever receives HTML or JSON, which is why you cannot "just show the data" without writing a query and a template.
3. **Each step can fail on its own.** When a page breaks, the first question is always *which step*. A 404 means step 2. A traceback in the terminal means step 3 or 4. A page that loads but shows nothing means step 5 came back empty.

The words you need

Word	What it actually means
Route	"When someone asks for this URL, run this function." Written <code>@main_bp.route('/about')</code> above a function.

View function	The function a route points at. It gathers data and returns a response. Most of your code goes here.
Model	A Python class that mirrors one database table. Our <code>Athlete</code> class has <code>first</code> and <code>last</code> fields, so the <code>athlete</code> table has those columns.
Query	The Python you write to ask for rows: filter, order, limit. SQLAlchemy turns it into SQL and runs it.
Template	An HTML file with blanks in it, filled in by Jinja: <code>{{ athlete.first }}</code> . Lives in <code>web/app/templates/</code> .

The one idea behind the ORM. You write this:

```
AthleteResult.query.filter_by(event="3200
Meters").order_by(AthleteResult.result2).limit(10).all()
```

SQLAlchemy turns it into roughly this and sends it to the database:

```
SELECT * FROM athlete_result WHERE event = '3200 Meters' ORDER BY result2 LIMIT 10;
```

The database answers with rows of text and numbers. SQLAlchemy turns each row back into a Python object, so `result.athlete.last` works. That is the entire trick. You are always writing SQL; you are just writing it in Python.

Part 1 — Git and GitHub: getting the code

Git is a time machine for a folder. It records snapshots of your project so you can see what changed, go back, and combine work from several people without anyone overwriting anyone else. GitHub is a website that stores a copy of that history so the team can share it.

Three words cover almost everything you will do:

- **Branch** — a parallel line of work. You make your changes on your own branch, so the working site is never affected by half-finished code.
- **Commit** — a saved snapshot with a message explaining why. Your unit of work.
- **Pull request (PR)** — a request to merge your branch into `main`, so a teammate can review it first.

The mental model that makes this click: `main` is the branch that becomes the live website. Nobody edits it directly. You branch off it, work, and ask to merge back.

Do it

All of this happens in VS Code, no terminal. Two spots do the work: the **Source Control** panel (the branching icon, or **Ctrl+Shift+G**) and the **branch name in the bottom-left status bar**. For anything you cannot find, press **Ctrl+Shift+P** and type **Git:**.

1. **Clone the project** (once, ever). Welcome screen → **Clone Git Repository**, or Command Palette → **Git: Clone**. Paste the URL your teacher gives you, pick a folder, open it.
2. **Start from the newest code**. Click the branch name in the status bar, pick **main**, then click the sync icon beside it.
3. **Create your branch**. Click the branch name again → **Create new branch...**, and type `training/your-name-top-3200`. Branch names are read by other people: `type/short-description`, lowercase, hyphens.
4. **Check where you are**. The status bar shows your new branch and Source Control shows no changes.

Look at that branch name before you start typing and before you commit. The number-one beginner mistake is doing an hour of good work on `main` by accident.

Part 2 — Running the app on your own machine

You are about to run a complete copy of Track Insights on your laptop. Nobody else can see it, and you can break it as thoroughly as you like. This part uses the terminal: open one in VS Code with **Ctrl+`** or **View → Terminal**, already pointed at your project folder.

Virtual environment

A virtual environment is a private folder of libraries belonging to this project only, so installing something here cannot break anything else on your computer.

```
# Create it (once per clone), from the project folder
python -m venv .venv

# Turn it on (every time you open a new terminal)
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # Mac

# Install everything the project needs (once per clone)
pip install -e ".[web,standalone,dev]"
```

When it is on, your prompt starts with `(.venv)`. If you ever see `ModuleNotFoundError`, you almost certainly forgot to activate it. Note there is no `requirements.txt` here: the library list lives in `pyproject.toml`, which is what that `pip install -e` line reads. The `-e` also installs our own `common/` package, which is why `from common.const import`

`CONST` works anywhere in the repo.

No configuration, no passwords

Most web projects make you set up a database and put its password somewhere. This one does not. `web/data/Track.db` is a SQLite database, one ordinary file, committed to the repository: you already have it, already full of real results. It is your own copy, so nothing you type here can reach the live site or the real data.

Start the server

```
cd web
python wsgi.py
```

You should see `Running on http://127.0.0.1:5000`. Open that in a browser and you have the whole site.

- `127.0.0.1` and `localhost` both mean *this computer*. Nobody else can reach it.
- Debug mode is already on: the server restarts itself whenever you save a `.py` or `.html` file, and errors show as a full traceback in the browser. CSS and JavaScript changes need a hard refresh, Ctrl+Shift+R.
- Leave that terminal running while you work. Every request prints a line there, and so does every error.

Part 3 — Read before you write

Do not skip this part. It does not feel like progress, and it is the difference between changing this codebase and guessing at it.

Find your bearings

Open the project in VS Code and find each of these. You do not have to read them yet, only know where they are.

Where	What lives there
<code>web/wsgi.py</code>	Starts the site. This is the file you run.
<code>web/app/models.py</code>	One Python class per database table
<code>web/app/routes/</code>	Which URL runs which function. <code>main_routes.py</code> for pages, <code>api_routes.py</code> for the JSON URLs
<code>web/app/queries/</code>	The longer database questions, one file per feature
<code>web/app/templates/</code>	The HTML. <code>base.html</code> is the shared shell every

<code>web/app/templates/</code>	The HTML <code>base.html</code> is the shared shell every page extends
<code>web/app/static/</code>	CSS, JavaScript, images
<code>web/data/Track.db</code>	The database itself
<code>docs/PROJECT_GUIDE.md</code>	The orientation doc for this repo. Read it.

The tracing exercise

Pick a page that already exists and follow it end to end. `/athlete-dashboard/101` is the useful one. Four questions:

1. Which function in `web/app/routes/main_routes.py` handles that URL? Search the file for it.
2. What does that function ask the database for? Say it in plain English.
3. What does it pass to `render_template`, which template gets it, and where is each value used?
4. Open the browser's Network tab (F12) and reload. Does the page also call an `/api/...` URL to fill itself in afterwards?

Do this *with* Codex rather than alone: ask it to walk you through the file. What you must not do is ask for a summary and move on without opening the files yourself.

Learn the data model

Open `web/app/models.py`. Six tables carry nearly everything on this site:

Table	Holds
<code>athlete</code>	One row per athlete: <code>first</code> , <code>last</code> , <code>school_id</code> , <code>gender</code> , <code>grad_year</code>
<code>school</code>	One row per school: <code>school_name</code> , <code>team_name</code> , <code>city</code>
<code>meet</code>	One row per meet: <code>meet_type</code> (Sectional, Regional, State), <code>gender</code> , <code>year</code> , <code>host</code>
<code>athlete_result</code>	One row per athlete, per event, per meet: <code>event</code> , <code>result</code> , <code>result2</code> , <code>place</code> , <code>grade</code>
<code>relay_result</code>	The same for relays, owned by a school rather than an athlete
<code>event</code>	The list of event names, and whether each is Track or Field

Seven facts about this data decide whether a query is right or quietly wrong. Learn them now:

- **result is for reading, result2 is for sorting.** `result` is the text a person sees, `8:47.57`. `result2` is the same mark as a number, `527.57` seconds. Filter and sort on `result2`, always.
- **Marks that are not marks are stored anyway.** `NT`, `DNF`, `DQ` and friends are real rows, with `result2` set to `9999` for times and `0` for distances. Leave them in and they sit at the top of your fastest list.
- **There is no gender column on athlete_result.** Gender lives on `athlete` and on `meet`, so a boys-only list needs a join to one of them.
- **Event names are spelled out in full.** `3200 Meters`, not `3200m`. Guess the spelling and you get zero rows.
- **result_type is Prelim or Final.** One athlete can have both at the same meet, so a careless query counts the same athlete twice.
- **We only have playoff meets, 2023 through 2026.** Sectional, Regional and State, no regular season. Nothing here is honestly "all-time".
- **Relays and field events are different.** Relays live in `relay_result` and belong to a school, not an athlete. Field-event marks are stored in inches, where bigger is better, so those sort the other way.

These are not trivia. Every one of them has put a wrong number on a page at some point.

Part 4 — Build the page

Build it in the order the request travels. After each step, reload the page in your browser. You want to see it fail in a new and more advanced way each time. That is progress.

Step 1 — The route and view function

Open `web/app/routes/main_routes.py`, go to the bottom, and add:

```
@main_bp.route('/top-3200')
def top_3200():
    return "Hello from the 3200 page"
```

Reload `http://127.0.0.1:5000/top-3200`. You should see the text.

Stop and understand what just happened, because this is the smallest complete version of the whole app:

- `@main_bp.route('/top-3200')` is a **decorator**. It adds the function below it to Flask's

URL table. It does not call the function.

- `main_bp` is a **blueprint**, a named group of routes: this project has `main_bp` for pages and `api_bp` for JSON. That is why the endpoint name is `main.top_3200`, which is how you will link to it later.
- `def top_3200():` is the **view function**, and whatever it returns becomes the page. Right now that is plain text. Next it will be HTML.

If you get a 404, Flask does not know about your route. Check the URL for typos, and check that the file saved: the terminal prints a restart line every time the server picks up a change.

Step 2 — The query

Now get real data. Add `AthleteResult` to the models imported at the top of the file, then fill in the function:

```
from ..models import Athlete, AthleteResult, School    # Athlete and School were
already here

@main_bp.route('/top-3200')
def top_3200():
    results = (
        AthleteResult.query
        .join(Athlete)
        .filter(AthleteResult.event == '3200 Meters')
        .filter(Athlete.gender == 'Boys')
        .filter(AthleteResult.result2 < 9999)
        .order_by(AthleteResult.result2)
        .limit(10)
        .all()
    )
    return f"Found {len(results)} results"
```

Read it as a sentence: from the results table, follow each row to the athlete who ran it, keep the rows whose event is 3200 Meters and whose athlete is a boy, drop the `NT` and `DNF` rows parked at 9999, sort by the number so the fastest is first, take the first ten, hand them back as a list.

`.join(Athlete)` needs no further explanation of *how* to join: `models.py` already says that `athlete_result.athlete_id` points at `athlete.athlete_id`, so SQLAlchemy works it out. You join because the gender you want to filter on is not on the result row; it is on the athlete.

One convention to know before a reviewer raises it: a short query like this is fine sitting in the route, and the athlete dashboard route does the same thing. Anything longer, reused

by a second page, or slow enough to want caching belongs in `web/app/queries/` instead, in the file for that feature.

The pieces you will reuse forever:

Method	What it does
<code>.filter(...)</code>	A WHERE clause. Chain several and they combine with AND.
<code>.order_by(X) / .desc()</code>	Sort. Ascending is fastest-first for times.
<code>.limit(n)</code>	Take only the first n rows.
<code>.all()</code>	Run the query, return a list.
<code>.first()</code>	Run it, return one object or <code>None</code> .

The critical detail: **nothing touches the database until you call `.all()`, `.first()` or `.count()`**. Everything before that is just building up a description of the query. This is why you can build a query in pieces across several lines of Python.

If you got `Found 0 results`, your filter values do not match what is stored. Do not guess at them. It is `3200 Meters`, not `3200m`, and `Boys`, not `B` or `M`.

Step 3 — Pass the data to a template

```
# replace the "Found N results" line with this:
return render_template('top-3200.html',
                       results=results,
                       title='Boys 3200 Meters - Top 10 Since 2023')
```

`render_template` finds that file in `web/app/templates/`, hands it the values you named, and returns the finished HTML. The name on the left of each `=` is the name the template will use. It is already imported at the top of `main_routes.py`, so there is nothing to add.

Step 4 — The template

Create `web/app/templates/top-3200.html`. Template names here are kebab-case, like the others in that folder. Copy the shape of an existing template rather than inventing one:

```
{% extends 'base.html' %}

{% block title %}Top 10 Boys 3200 - Track Insights{% endblock %}
```

```

{% block content %}
<div class="min-h-screen bg-white pt-20 sm:pt-24 pb-24 px-2 sm:px-4">
  <div class="max-w-3xl mx-auto">
    <h1 class="font-impact italic text-primary text-2xl sm:text-3xl font-bold mb-6">
      {{ title }}</h1>

    {% if results %}
    <div class="overflow-x-auto">
      <table class="table">
        <thead>
          <tr><th>#</th><th>Athlete</th><th>School</th><th>Time</th><th>Meet</th>
</tr>
        </thead>
        <tbody>
          {% for r in results %}
          <tr>
            <td>{{ loop.index }}</td>
            <td>{{ r.athlete.first | title }} {{ r.athlete.last | title }}</td>
            <td>{{ r.athlete.school.school_name }}</td>
            <td>{{ r.result }}</td>
            <td>{{ r.meet.meet_type }} {{ r.meet.year }}</td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
    </div>
    {% else %}
    <p>No results found.</p>
    {% endif %}
  </div>
</div>
{% endblock %}

```

Four pieces of Jinja syntax do nearly everything:

- `{{ ... }}` prints a value into the HTML.
- `{% ... %}` is logic: `for`, `if`, `extends`, `block`. It controls what gets printed but prints nothing itself.
- `{% extends %}` and `{% block %}` are inheritance. Your template fills a hole in `base.html` and inherits the header, nav, CSS and footer for free. This is why your page will already look like the rest of the site.
- `loop.index` is the 1-based counter inside a `for` loop. Free numbering.

Three details in there that are specific to this site:

- `| title` fixes the capitals. Names are stored shouting, `NOAH BONTRAGER`, because that is how the results come in. `| title` is a Jinja filter, a small function applied to a

that is how the results come in. `| title` is a Jinja filter, a small function applied to a value on its way onto the page.

- **pt-20 sm:pt-24 is not decoration.** The nav bar is pinned to the top of the window, so without that padding your heading hides underneath it.
- **The class names are Tailwind and DaisyUI**, and they only do anything if they are already in the built stylesheet `web/app/static/css/output.css`. That file is generated from `input.css`; never edit it by hand. Reuse classes that other pages already use and you will not have to rebuild anything.

Notice `r.athlete.school.school_name`. You never asked for the school; SQLAlchemy followed the link from result to athlete to school. Convenient, and a trap: inside a loop that fires an extra query per row. Ten rows is fine. If you ever loop over a thousand, ask Codex about `joinedload`.

Step 5 — Link it into the site

A page nobody can reach does not exist. The top nav bar is full, so put yours on the Insights page instead: open `web/app/templates/insights/index.html`, copy one of the cards in the grid, and change its link, heading and description.

```
<!-- copy a whole <a> card and change these two lines -->
<a href="{% url_for('main.top_3200') %}" class="...same classes as the card
above...">
<h2 class="text-base font-semibold group-hover:text-primary transition-colors">Top
10 Boys 3200</h2>
```

Use `url_for('main.top_3200')` rather than typing `/top-3200`. The name is the blueprint plus the function name, which is why it is `main.` and not just `top_3200`. If the URL ever changes, every `url_for` link updates itself and every hand-typed link breaks silently.

Part 5 — Check your work

A page that loads is not a page that is correct. On a data site, a confidently wrong number is worse than an error, because nobody catches it. Coaches and athletes will read this.

Verify the data by hand

Take the top three results your page shows and look them up independently against the real meet results. If one of them is wrong, your query is wrong, and you now know it before a reviewer does.

Then go looking for trouble:

□ Are the times in order, fastest first? That works because you sorted on `result+2`. As

- ☐ Are the times in order, fastest first? That works because you sorted on `result2`. As text, `10:01.22` would come before `9:42.10`.
- ☐ Did an `NT`, `DNF` or `DQ` row sneak in at the top as a suspiciously fast time?
- ☐ Are any girls' results in the boys' list, or the same athlete listed twice from a prelim and a final?
- ☐ Is one athlete taking several of the ten spots? Noah Bontrager legitimately holds two. Ten fastest *performances* or ten fastest *athletes*? Say which in your pull request.
- ☐ Does the page look like the rest of the site, and work at phone width?
- ☐ Nothing red in the terminal, nothing red in the browser console (F12).

Write an automated test

You get one test for free: `tests/test_routes_smoke.py` walks every route Flask knows about and checks that each one answers, so your page is covered the moment you add it. Write one of your own too, in `tests/test_top_3200.py`:

```
def test_top_3200_page_loads(client):
    response = client.get('/top-3200')
    assert response.status_code == 200
    assert b'3200' in response.data
```

Run the suite with `python -m pytest tests/ -q`. That `client` fixture in `tests/conftest.py` builds a small made-up database instead of using `Track.db`, which is why the test checks that the page works, not that Noah Bontrager is first. One habit worth keeping: write the test *before* you ask Codex for the code. Deciding what "correct" means is the part of the job that is actually yours.

Part 6 — Commit, push, and open a pull request

Saving your work

Git has a two-step save, which seems fussy until you need it. **Staging** is choosing which changes go into this snapshot. **Committing** takes the snapshot. The gap between them lets you commit two unrelated fixes separately instead of mashing them together.

1. **Review what changed.** In the Source Control panel, click each filename under **Changes**. VS Code opens a side-by-side diff: the old version on the left, yours on the right. Read all of it.
2. **Stage what you meant to change.** Hover a file and click **+**, one file at a time rather than the **+** on the *Changes* header, which sweeps up everything.
3. **Commit.** Type a message in the box at the top, then press **Ctrl+Enter**.
4. **Send it to GitHub.** The button below the message box reads **Publish Branch** the first

time and **Sync Changes** after that.

Always open the diff for every file before committing. With an AI writing code, this is the single most important habit in this document. It is the moment you find out that Codex also reformatted a file you never asked it to touch, or deleted a function it decided was unused.

Commit messages

Write what the change does, in the imperative, in one line under about 70 characters:

Add top 10 boys 3200m page, not updates. Commit several times as you work rather than once at the end — each commit is a point you can return to.

The pull request

On github.com, click the **Compare & pull request** banner that appears after you publish a branch. Write a description that answers three questions:

1. **What does this do?** One or two sentences.
2. **How did you verify it?** Name the actual checks. "Confirmed the top three against the 2025 State meet results" is a real answer; "it works" is not.
3. **What should the reviewer look at?** Anything you are unsure about.

Include a screenshot of the page. Pushing also runs the test suite on GitHub: a red X on your pull request means `pytest` failed there, and clicking it shows which test. Fix that before asking anyone to review.

A teammate then reads your diff and asks you to explain lines of your own code. That is not a gotcha: it is how the team shares knowledge, and the last check before something wrong goes in front of real users. **If you cannot explain what a line does, it does not get merged**, regardless of who or what wrote it. Go read it, ask Codex to explain it, come back. To make changes after review, commit and push to the same branch; the pull request updates itself.

Part 7 — After the merge

Once your pull request is merged, your code is part of `main`, and `main` is what the live site runs. Two things differ there. Debug mode is off, so an error a visitor hits shows a plain page rather than a traceback. And some pages, the school dashboards especially, are served from prepared answers in `dashboard_cache.db` and `web/app/static/data/`, which are rebuilt by hand after results change — which is also why a school dashboard can look empty on a fresh clone. The data itself is the same as yours, because `Track.db` is in the repository. Check your page on the live site once it deploys, and on your phone.

Clean up and go again

1. Click the branch name in the status bar and switch to **main**.
2. Click **Sync Changes** to pull in the merged work, including yours.
3. Open the Command Palette, run **Git: Delete Branch...**, and pick your finished branch.

Deleting the branch loses nothing: your commits are in `main` now. Your next task is the same three moves every time: switch to main, sync, create a branch. Always branch from a freshly synced main, because branching from a stale one is how you end up with merge conflicts.

Working with Codex

Codex will write most of the code on this project, and that is fine. It changes what your job is, not whether you have one. Your job is deciding what to build, judging whether what came back is right, and finding out why it is not.

Four rules

1. **You cannot merge code you cannot explain.** Line by line, out loud, to a teammate. Everything else rests on this one.
2. **Read every diff before you commit.** Codex edits files you did not ask it about. Catch that in the Source Control panel, not on the live site.
3. **Debug for ten minutes before you paste the error.** Make a guess, add a `print`, look at the actual data. Then ask for help. Finding where a bug lives is the skill that transfers to everything else you will ever do, and it is the first one to disappear if you hand it over immediately.
4. **Never trust it on track and field, or on this database.** It does not know that our events are called `3200 Meters`, that `NT` is stored as 9999, that we only hold playoff meets since 2023, that the 1600 is not the mile, or that hand times and FAT are not interchangeable. It will write confident, tidy, wrong code in exactly those places. Check it against real results.

Prompting that works

Vague requests get generic code that does not fit this codebase. "Make a page showing the fastest 3200 times" is weak. This is not:

```
In web/app/routes/main_routes.py, following the same pattern as the
athlete_dashboard route, add a route at /top-3200 that returns the 10 fastest
boys' 3200 Meters results. Use the AthleteResult and Athlete models in
web/app/models.py. Sort ascending on result2 and exclude rows where result2
```

`web/app/models.py`: sort ascending on `result2` and exclude rows where `result2` is 9999, which is how we store `NT` and `DNF`. Render `templates/top-3200.html`, which should extend `base.html` and use the same table markup as the other list pages. Then explain the query to me line by line.

It names the files, points at an existing pattern to copy, states the edge case, and asks for an explanation. Asking for the explanation is not politeness; it is how you get what you need to pass code review. Two things worth asking every time:

- "Explain this line by line." Then actually read it.
- "What could go wrong with this? What edge cases does it not handle?" Codex is often good at critiquing code, including its own.

Self-check, troubleshooting, and commands

Can you answer these out loud?

If you can answer these, you are ready for real tasks. If one of them stumps you, go back to that part.

1. Someone opens `trackinsights.org/top-3200`. Walk through everything that happens until they see the page.
2. What is the difference between `result` and `result2`, and why do you sort on the second one?
3. What is stored in the row where an athlete ran `NT`, and what happens if you forget about it?
4. What is the difference between `{{ }}` and `{% %}` in a template?
5. Why does your page already have the site header and nav without you writing one?
6. Your page loads but the table is empty. Name three possible causes and how you would tell them apart.

When something breaks

What you see	Usually means
<code>404 Not Found</code>	Flask has no route for that URL. Typo, or the file never saved.
<code>500 Internal Server Error</code>	Your Python raised an exception. The real message is in the terminal running the server. Read it bottom-up.
<code>TemplateNotFound</code>	The filename in <code>render_template()</code> does not

	match the file in <code>templates/</code> .
<code>jinja2.UndefinedError</code>	The template uses a variable the view function never passed, or misspelled it.
<code>ModuleNotFoundError</code>	The virtual environment is not switched on, or you have not run the <code>pip install -e</code> line in this clone.
Page loads, table empty	The query matched zero rows. Print <code>len(results)</code> and check the actual values in the column you filtered on.
<code>CONFLICT</code> during merge	Two people changed the same lines. VS Code marks the spots and gives you Accept Current / Accept Incoming buttons. Ask for help the first time.

How to read a Python traceback: start at the bottom. The last line is the actual error. The lines above it are the path the program took to get there. Find the topmost line that names a file *you* wrote; that is usually where your bug is.

Cheat sheet

Git, in the VS Code GUI

To do this	Where
Open Source Control	Branch icon in the Activity Bar, or Ctrl+Shift+G
See or switch branch	Click the branch name, bottom-left of the status bar
Create a branch	Click the branch name → Create new branch...
See what I changed	Click a filename under Changes → side-by-side diff
Stage, then commit	Hover the file → + , type the message, Ctrl+Enter
Push	Publish Branch the first time, Sync Changes after
Anything not listed	Ctrl+Shift+P , then type <code>Git:</code>

On a Mac, **Ctrl** is **Cmd** in all of the above.

Still the terminal (these are Python, not Git)

```
.venv\Scripts\activate
Mac)
cd web
```

```
# Windows (source .venv/bin/activate on
```

```
python wsgi.py           # start the site on localhost:5000
python -m pytest tests/ -q # run the tests, from the project folder
pip install -e ".[web,standalone,dev]" # after someone adds a library
```